

Quantum Machine Learning: A Practical Introduction

From Quantum Circuits to Hybrid Neural Networks

Jacob Cybulski

Sebastian Zając

2026

Table of contents

Preface	1
What This Book Is About	1
Prerequisites	1
Software Setup	1
How This Book Is Organised	2
Notation Conventions	2
Source Material	2
 I Part I: Foundations	 3
1 Quantum Circuits and PennyLane Basics	4
1.1 What Is a Quantum Circuit?	4
1.2 PennyLane in Five Minutes	4
1.2.1 Devices	4
1.2.2 Quantum Nodes (QNodes)	5
1.2.3 Rotation Gates	5
1.3 Measurement Types	5
1.3.1 Probability Measurement	5
1.3.2 Expectation Value	6
1.4 Circuit Visualisation	6
1.5 Computing Gradients	6
1.5.1 Jacobian Example	7
1.6 Parameter Broadcasting (Batching)	7
1.7 A Minimal QML Model	7
1.8 Exercises	7
 2 Data Encoding Strategies	 9
2.1 Why Encoding Matters	9
2.2 Amplitude Embedding	9
2.3 Basis Embedding	10
2.4 Angle Embedding	10
2.4.1 Recovering the Input from Measurements	11
2.5 Encoding Margin (Scaling)	11
2.6 Data Reuploading	11
2.7 Comparison Summary	12
2.8 Exercises	12

3	Anatomy of a Hybrid Model	13
3.1	The Core Idea	13
3.2	Connecting PennyLane to PyTorch	13
3.2.1	Step 1 — Define the QNode with <code>interface="torch"</code>	13
3.2.2	Step 2 — Wrap as a <code>TorchLayer</code>	14
3.2.3	Step 3 — Embed in an <code>nn.Module</code>	14
3.3	Multi-Branch Hybrid Architecture	14
3.4	Training Loop	15
3.5	Weight Shape Reference	15
3.6	Key Considerations	15
3.7	Exercises	16
II	Part II: Practical Models	17
4	Your First Quantum Regression Model	18
4.1	Problem Setup	18
4.2	Anatomy of a Variational QML Model	18
4.3	Cost vs Loss	19
4.4	Model 1 — Basic Entangler (Simplest)	19
4.4.1	Training	19
4.5	Model 2 — Strong Entangler (More Expressive)	19
4.6	Model 3 — Data Reuploading (Most Expressive)	20
4.7	Optimiser Comparison	20
4.8	Initialisation Matters	21
4.9	Evaluating Fit Quality	21
4.10	Exercises	21
5	Quantum Estimator for Real Datasets	23
5.1	What Is a Quantum Estimator?	23
5.2	Datasets	23
5.3	Preprocessing	23
5.4	The Estimator Circuit	24
5.5	Train / Test Split	24
5.6	Training	25
5.7	Evaluation Metrics	25
5.7.1	Mean Squared Error	25
5.7.2	R^2 (Coefficient of Determination)	25
5.8	Residuals Analysis	25
5.9	Ansatz Comparison	26
5.10	Exercises	26
6	Quantum Classifier	28
6.1	Dataset: Automobile Insurance Risk	28
6.2	Dimensionality Reduction	29
6.3	The Quantum Circuit	29
6.4	Quantum Classifier Model	29
6.5	Training	30

6.6	Evaluation	31
6.7	Making Predictions on New Data	31
6.8	Key Design Choices	31
6.9	Exercises	32
7	Classical Baseline Classifier	33
7.1	Why a Baseline Matters	33
7.2	Classical Classifier Architecture	33
7.3	Training (Same Setup as Quantum)	34
7.4	Evaluation	34
7.5	Side-by-Side Comparison	35
7.6	Interpreting the Results	35
7.7	When Might Quantum Win?	36
7.8	Exercises	36
III	Part III: Advanced Topics	37
8	Hybrid Quantum-Classical Logistic Regression	38
8.1	From Linear to Quantum Score	38
8.2	Binary Cross-Entropy Loss	38
8.3	The Quantum Circuit	39
8.4	Model: <code>expval</code> Path	39
8.5	Model: <code>probs</code> Path (Alternative)	39
8.6	Training	40
8.7	Adding a Classical Booster Layer	40
8.8	LBFSG Optimiser (Advanced)	41
8.9	Exercises	41
9	Quantum Autoencoder: Pure PennyLane	43
9.1	The Problem: Denoising Time Series	43
9.2	Sliding Windows	44
9.3	QAE Architecture	44
9.4	SWAP Test for State Comparison	44
9.5	QAE Circuit	45
9.6	Loss Function	46
9.7	Training Loop	46
9.8	Evaluation	47
9.9	Exercises	47
10	Hybrid and Mirror QAE	48
10.1	Hybrid QAE with PyTorch	48
10.1.1	Circuit as <code>TorchLayer</code>	48
10.1.2	Hybrid Model	49
10.2	Mirror QAE: Three-Stage Training	49
10.2.1	Stage 1 — Train the Decoder on Clean Data	50
10.2.2	Stage 2 — Train the Encoder on Noisy Data	50
10.2.3	Stage 3 — Joint Fine-Tuning (Optional)	51
10.3	Why Phase Information Matters	51

10.4 Comparison: Pure vs Hybrid vs Mirror QAE	51
10.5 Exercises	52
IV Appendices	53
11 PennyLane API Reference	54
11.1 Devices	54
11.2 QNode Decorator	54
11.3 Single-Qubit Gates	54
11.4 Two-Qubit Gates	55
11.5 Data Encoding	55
11.6 Ansatz Templates	55
11.7 Measurements	55
11.8 Visualisation	56
11.9 Differentiation	56
11.10 PyTorch Integration	56
11.11 Native Optimisers (PennyLane only)	57
12 Mathematical Background	58
12.1 Linear Algebra Essentials	58
12.1.1 Vectors and Inner Products	58
12.1.2 Tensor Products	58
12.1.3 Density Matrices	59
12.2 Quantum Gates as Matrices	59
12.2.1 Rotation Gates	59
12.3 Measurements and Observables	59
12.3.1 Expectation Value	59
12.3.2 Pauli Operators	60
12.4 Variational Principle	60
12.5 Parameter-Shift Rule	60
12.6 Loss Functions	60
12.6.1 Mean Squared Error (MSE)	60
12.6.2 R^2 (Coefficient of Determination)	60
12.6.3 Binary Cross-Entropy (BCE)	61
12.6.4 Sigmoid	61
12.7 SWAP Test	61
12.8 Mackey-Glass Equation	61
13 Datasets	62
13.1 UCI Automobile Dataset	62
13.2 Mackey-Glass Time Series	63
13.3 Scikit-learn Synthetic Datasets	64
13.3.1 <code>make_regression</code>	65
13.3.2 <code>make_swiss_roll</code>	65
References	66

Preface

This book is a companion to the *QML Workshop Intro* — a hands-on introduction to Quantum Machine Learning (QML) developed for the QPoland and QAIF communities.

What This Book Is About

Quantum Machine Learning sits at the intersection of quantum computing and classical machine learning. Rather than surveying the theoretical landscape, this book takes a **practice-first** approach: every concept is introduced through working Python code using [PennyLane](#) and [PyTorch](#).

By the end you will be able to:

- Build and train variational quantum circuits for regression and classification
- Connect quantum circuits to PyTorch’s autograd engine via `qml.qnn.TorchLayer`
- Understand the key design choices: data encoding strategy, ansatz architecture, measurement type, and optimizer
- Implement a Quantum Autoencoder (QAE) for time-series denoising

Prerequisites

Topic	Required level
Python	Comfortable with NumPy, functions, classes
Machine learning	Knows loss functions, gradient descent, train/test split
Linear algebra	Vectors, matrix multiplication, inner products
Quantum computing	<i>None required</i> — introduced from scratch

Software Setup

All code runs on Python 3.11+ with a virtual environment:

```
python3.11 -m venv qml_venv
source qml_venv/bin/activate      # macOS / Linux
# .\qml_venv\Scripts\activate     # Windows
pip install -r requirements.txt
```

Key packages used throughout:

Package	Version	Role
pennylane	0.42.3	Quantum circuit definition and differentiation
pennylane-lightning	0.42.0	Fast CPU/GPU simulator
torch	2.8.0	Automatic differentiation, hybrid layers
scikit-learn	1.7.2	Datasets, preprocessing, metrics
matplotlib	3.10.6	Visualization

How This Book Is Organised

Part I — Foundations covers quantum circuits, data encoding strategies, and the anatomy of a hybrid model. This material corresponds to the self-study *Explore* sessions of the workshop and can be read at your own pace.

Part II — Practical Models builds three complete models: a quantum regressor, a quantum estimator for real datasets, and a quantum classifier compared against a classical baseline.

Part III — Advanced Topics introduces hybrid quantum-classical logistic regression and two variants of the Quantum Autoencoder for time-series denoising.

Appendices provide a PennyLane API reference, a mathematical notation guide, and dataset descriptions.

Notation Conventions

Throughout the book we use Dirac notation:

- $|0\rangle, |1\rangle$ — computational basis states
- $|\psi\rangle$ — general quantum state
- $\langle\psi|\phi\rangle$ — inner product
- $\hat{O}$ — observable operator
- $\langle\hat{O}\rangle$ — expectation value

Code comments follow the pattern `# Compute = | |` to connect math and code.

Source Material

This book is derived from workshop notebooks originally authored by Jacob Cybulski (Enquanted), Tomasz Rybotycki, Sebastian Zając, and Paweł Góra, and released under the [GNU GPL v3](#) licence.

The original notebooks are available at: https://github.com/ironfrown/qml_workshop_intro_v3

Part I

Part I: Foundations

Chapter 1

Quantum Circuits and PennyLane Basics

1.1 What Is a Quantum Circuit?

A quantum circuit is a sequence of operations — called *gates* — applied to one or more *qubits*. Unlike classical bits, which are always 0 or 1, a qubit can be in a **superposition** of both states:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1$$

The coefficients α and β are complex amplitudes. When we *measure* the qubit, the superposition collapses: we observe $|0\rangle$ with probability $|\alpha|^2$ and $|1\rangle$ with probability $|\beta|^2$.

i Note

The requirement $|\alpha|^2 + |\beta|^2 = 1$ is the *normalisation condition*. It ensures that the total probability of all outcomes sums to one.

1.2 PennyLane in Five Minutes

[PennyLane](#) is a Python library for differentiable quantum programming. Its key idea: quantum circuits are treated as differentiable functions, so gradients can be computed and used by any optimiser.

1.2.1 Devices

A *device* is the (simulated or real) quantum hardware that executes circuits:

```
import pennylane as qml

# Fast CPU simulator (default)
dev = qml.device("default.qubit", wires=2)
```

```
# Lightning C++ simulator - much faster for >10 qubits
dev_fast = qml.device("lightning.qubit", wires=2)
```

wires specifies the number of qubits. Wire indices are integers 0, 1, 2,

1.2.2 Quantum Nodes (QNodes)

A *QNode* wraps a Python function that builds a circuit:

```
@qml.qnode(dev)
def my_circuit(theta):
    qml.RY(theta, wires=0)          # rotate qubit 0 by angle theta
    qml.CNOT(wires=[0, 1])         # entangle qubits 0 and 1
    return qml.probs(wires=[0, 1]) # measure all 4 outcome probabilities
```

Calling `my_circuit(0.5)` executes the circuit and returns a NumPy array of probabilities $[P_{00}, P_{01}, P_{10}, P_{11}]$.

1.2.3 Rotation Gates

The most common single-qubit gates are the *Pauli rotations*:

$$R_Y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$$

Applied to $|0\rangle$, $R_Y(\theta)$ produces the state $\cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle$, so:

$$P(|0\rangle) = \cos^2(\theta/2), \quad P(|1\rangle) = \sin^2(\theta/2)$$

```
@qml.qnode(dev)
def ry_circuit(theta):
    qml.RY(theta, wires=0)
    return qml.probs(wires=0)

import numpy as np
theta = np.pi / 3
probs = ry_circuit(theta)
# probs[0]   cos^2( /6)   0.75
# probs[1]   sin^2( /6)   0.25
```

1.3 Measurement Types

PennyLane supports two primary measurement strategies.

1.3.1 Probability Measurement

```
return qml.probs(wires=[0, 1])
# → array of 2^n probabilities, shape (4,) for 2 qubits
```

Returns the full probability distribution over all 2^n computational basis states.

1.3.2 Expectation Value

```
return qml.expval(qml.PauliZ(0))
# → scalar in [-1, +1]
```

The Pauli-Z expectation value is:

$$\langle Z \rangle = P(|0\rangle) - P(|1\rangle)$$

It maps to +1 when the qubit is certainly $|0\rangle$ and -1 when certainly $|1\rangle$. This scalar output is what most QML models use as a prediction.

! Important

probs vs expval — output shape difference. A circuit returning `qml.probs()` produces one probability vector per call, even when inputs are batched. A circuit returning `qml.expval()` produces one scalar per qubit per sample when parameter broadcasting is used.

1.4 Circuit Visualisation

```
print(qml.draw(my_circuit)(0.5))
qml.draw_mpl(my_circuit)(0.5) # matplotlib figure
```

Barriers can be added for visual clarity without affecting circuit behaviour:

```
qml.Barrier(wires=[0, 1])
```

1.5 Computing Gradients

PennyLane can differentiate circuits with respect to their parameters. The default method for most circuits is the **parameter-shift rule**:

$$\frac{\partial f}{\partial \theta} = \frac{f(\theta + \pi/2) - f(\theta - \pi/2)}{2}$$

This requires exactly two circuit evaluations per parameter. For circuits that support it, the **adjoint** method is faster (one forward + one backward pass):

```
@qml.qnode(dev, diff_method='adjoint')
def circuit(params, x):
    ...
```

1.5.1 Jacobian Example

For a circuit with multiple parameters, the full Jacobian shows how each output probability changes with each parameter:

```
# Jacobian: shape (n_outputs, n_params)
jac = qml.jacobian(my_circuit)(params)
```

1.6 Parameter Broadcasting (Batching)

PennyLane can evaluate a circuit on an entire batch of inputs at once:

```
X_batch = np.array([0.1, 0.5, 1.2, 2.0]) # 4 inputs

@qml.qnode(dev)
def circuit(x):
    qml.RY(x, wires=0)
    return qml.expval(qml.PauliZ(0))

results = circuit(X_batch) # shape (4,) - one scalar per input
```

This avoids a Python loop and is significantly faster on simulators.

1.7 A Minimal QML Model

Here is the smallest possible variational circuit — one qubit, one trainable parameter, one input feature:

```
dev = qml.device("default.qubit", wires=1)

@qml.qnode(dev, diff_method='adjoint')
def tiny_model(params, x):
    # Feature map: encode input x
    qml.RY(x, wires=0)
    # Ansatz: trainable rotation
    qml.RY(params[0], wires=0)
    # Measurement
    return qml.expval(qml.PauliZ(0))
```

The circuit has two rotation layers: one controlled by data (`x`) and one by a learnable parameter. Training adjusts `params[0]` to minimise a loss function.

1.8 Exercises

1. Build a 2-qubit circuit that applies `qml.RY(theta, wires=0)` followed by `qml.CNOT(wires=[0, 1])`. Measure `qml.probs(wires=[0, 1])`. What are the probabilities when `theta = 0`? When `theta = ?`
2. Change the measurement to `qml.expval(qml.PauliZ(0))`. How does the output differ from `probs`?

3. Compute the gradient of the `tiny_model` above with respect to `params[0]` using `qml.grad`. Verify manually using the parameter-shift rule formula.
4. Run `tiny_model` on a batch of 8 equally spaced inputs in $[0, \pi]$. Plot the output as a function of x .

Chapter 2

Data Encoding Strategies

Encoding classical data into a quantum state is one of the most consequential design decisions in any QML pipeline. The choice of encoding affects expressibility, trainability, and the number of qubits required.

This chapter covers the three encoding strategies used throughout this book: **Amplitude Embedding**, **Basis Embedding**, and **Angle Embedding**.

2.1 Why Encoding Matters

Classical ML feeds a vector $\mathbf{x} \in \mathbb{R}^d$ directly into a model. Quantum ML must first map $\mathbf{x}$ to a quantum state $|\psi(\mathbf{x})\rangle$. Different mappings give the quantum circuit access to different representations of the data, and some are exponentially more compact than others.

2.2 Amplitude Embedding

Idea: Encode a normalised vector of 2^n values as the amplitudes of an n -qubit state.

$$|\psi\rangle = \sum_{i=0}^{2^n-1} x_i |i\rangle, \quad \|\mathbf{x}\| = 1$$

This is exponentially compact: n qubits can encode 2^n values.

```
import pennylane as qml
import numpy as np

dev = qml.device("default.qubit", wires=2)  # 2 qubits → 4 amplitudes

@qml.qnode(dev)
def amplitude_circuit(x):
    # x must be normalised to unit norm
    qml.AmplitudeEmbedding(x, wires=[0, 1], normalize=True)
    return qml.state()
```

```
x = np.array([1.0, 2.0, 3.0, 4.0])
state = amplitude_circuit(x)
# state [0.182, 0.365, 0.548, 0.730] (normalised x)
```

Warning

AmplitudeEmbedding requires the input vector to have exactly 2^n elements. Use `normalize=True` to handle unnormalised inputs automatically.

Limitation: Preparing an arbitrary amplitude state is generally hard (requires $O(2^n)$ gates). This encoding is most useful when the state preparation itself is tractable, or when we know the data has special structure.

2.3 Basis Embedding

Idea: Encode a binary string $\mathbf{b} \in \{0,1\}^n$ as the corresponding computational basis state $|b_0 b_1 \dots b_{n-1}\rangle$.

```
@qml.qnode(dev)
def basis_circuit(bits):
    # bits: integer list, e.g. [1, 0] → |10
    qml.BasisEmbedding(bits, wires=[0, 1])
    return qml.state()

state = basis_circuit([1, 0])
# state = [0, 0, 1, 0] → |10
```

Each qubit is flipped with an X gate wherever $b_i = 1$. This encoding is deterministic and requires no normalisation.

Limitation: Only encodes binary data. Continuous features must be discretised first. Each sample requires exactly n bits.

2.4 Angle Embedding

Idea: Encode each feature x_i as the rotation angle of a single-qubit gate. This is the most widely used encoding in variational QML.

$$\text{AngleEmbedding}(\mathbf{x}) = \bigotimes_{i=0}^{n-1} R_Y(x_i)$$

```
dev = qml.device("default.qubit", wires=3)

@qml.qnode(dev)
def angle_circuit(x):
    # x: array of shape (n_qubits,)
    qml.AngleEmbedding(x, wires=[0, 1, 2], rotation='Y')
    return qml.probs(wires=[0, 1, 2])
```

```
x = np.array([0.5, 1.2, 0.3])
probs = angle_circuit(x)
```

The rotation axis can be 'X', 'Y', or 'Z'.

2.4.1 Recovering the Input from Measurements

For a single qubit with $R_Y(\theta)$ applied to $|0\rangle$:

$$P(|0\rangle) = \cos^2(\theta/2)$$

Inverting: $\theta = 2 \arccos(\sqrt{P(|0\rangle)})$. This means the original amplitude $\cos(\theta/2)$ can be approximately recovered from measurement statistics — though this requires many shots.

2.5 Encoding Margin (Scaling)

When using angle encoding with real-valued features, it is common practice to scale inputs to a fraction of π :

```
x_angle_margin = 0.5 # use at most /2 of the rotation range

@qml.qnode(dev, diff_method='adjoint')
def circuit_with_margin(params, x):
    # x is already normalised to [-1, 1]
    qml.RY(x * np.pi * x_angle_margin, wires=0)
    ...
```

A small margin (e.g., 0.3) keeps features away from the poles of the Bloch sphere, where gradients can vanish. A larger margin (e.g., 0.5) gives more expressive power but may introduce barren plateau issues for deep circuits.

2.6 Data Reuploading

A single encoding layer limits the circuit's expressibility: the feature map is applied only once. **Data reuploading** alternates encoding and trainable layers:

$R_Y(x) \rightarrow R_Y(\) \rightarrow R_Y(x) \rightarrow R_Y(\) \rightarrow \dots \rightarrow \text{measurement}$

This is analogous to a deep classical network where the input is re-injected at multiple layers. It allows the circuit to implement more complex functions of the input.

```
@qml.qnode(dev, diff_method='parameter-shift')
def reuploading_circuit(params, x):
    n_layers = params.shape[0]
    for l in range(n_layers):
        # Re-encode data at every layer
        qml.AngleEmbedding(x, wires=range(n_qubits), rotation='Y')
        # Trainable ansatz
```



```
qml.BasicEntanglerLayers(params[l:l+1], wires=range(n_qubits))
return qml.expval(qml.PauliZ(0))
```

i Note

`diff_method='adjoint'` does not support data reuploading (mid-circuit encoding layers break the adjoint pass). Use `'parameter-shift'` instead, at the cost of more circuit evaluations.

2.7 Comparison Summary

Strategy	Qubits for d features	Notes
Amplitude	$\lceil \log_2 d \rceil$	Exponentially compact; hard state prep
Basis	d bits	Binary data only
Angle	d qubits	Most practical; one qubit per feature
Angle + reuploading	d qubits	Higher expressibility; slower

2.8 Exercises

1. Create a 3-qubit `AngleEmbedding` circuit using `rotation='X'`. Measure `qml.expval(qml.PauliZ(i))` for each qubit i . How does the expectation value relate to the input angle x_i ?
2. For `AmplitudeEmbedding` with 4 qubits, what is the maximum number of features that can be encoded? What happens if you pass a vector of length 10?
3. Implement a 2-layer reuploading circuit on 2 qubits with `StronglyEntanglingLayers` as the ansatz. Compare its output to a single-layer (no reuploading) circuit on a sine wave dataset.
4. Compute the encoding margin that maps the feature range $[-3, 3]$ to the rotation range $[-\pi/2, \pi/2]$. Write the corresponding scaling code.

Chapter 3

Anatomy of a Hybrid Model

A *hybrid quantum-classical model* combines quantum circuits with classical neural network layers. Understanding how these pieces connect is essential before building full models in Part II.

3.1 The Core Idea

Classical layers and quantum layers are interchangeable components in PyTorch’s computation graph. The quantum circuit acts as a differentiable function — its Jacobian with respect to trainable parameters is computed by PennyLane and passed back through PyTorch’s autograd engine.

Input data

[Classical preprocessing] $\leftarrow$ nn.Linear, nn.ReLU, ...

[Quantum circuit] $\leftarrow$ @qml.qnode(dev, interface="torch")

[Classical postprocessing] $\leftarrow$ nn.Linear, nn.Sigmoid, ...

Output (prediction)

3.2 Connecting PennyLane to PyTorch

3.2.1 Step 1 — Define the QNode with interface="torch"

```
import pennylane as qml
import torch
import torch.nn as nn

n_qubits = 2
```

```

n_layers = 2
dev = qml.device("default.qubit", wires=n_qubits)

@qml.qnode(dev, interface="torch", diff_method="adjoint")
def quantum_circuit(inputs, weights):
    # inputs: shape (n_qubits,)
    # weights: shape (n_layers, n_qubits, 3)
    qml.AngleEmbedding(inputs, wires=range(n_qubits), rotation='Y')
    qml.StronglyEntanglingLayers(weights, wires=range(n_qubits))
    return qml.expval(qml.PauliZ(0))

```

3.2.2 Step 2 — Wrap as a TorchLayer

`qml.qnn.TorchLayer` turns the `QNode` into a standard `nn.Module` layer:

```

weight_shapes = {"weights": (n_layers, n_qubits, 3)}
quantum_layer = qml.qnn.TorchLayer(quantum_circuit, weight_shapes)

```

The `weight_shapes` dictionary maps each trainable parameter name to its shape. PennyLane initialises these weights randomly and registers them with PyTorch so they appear in `model.parameters()`.

3.2.3 Step 3 — Embed in an `nn.Module`

```

class SimpleHybrid(nn.Module):
    def __init__(self):
        super().__init__()
        self.quantum = quantum_layer          # quantum layer
        self.out      = nn.Linear(1, 1)       # classical output layer

    def forward(self, x):
        # x: shape (batch, n_qubits)
        q_out = self.quantum(x).float()       # shape (batch, 1)
        return self.out(q_out)               # shape (batch, 1)

```

3.3 Multi-Branch Hybrid Architecture

Some problems benefit from processing different parts of the input with separate branches — either quantum or classical — before joining them:

```

# f(d1, d2) = m1( c3( join( c1(d1), c2( f2(d2) ) ) ) )
class MultiBranchHybrid(nn.Module):
    def __init__(self):
        super().__init__()
        # Classical pre-processing for branch 2
        self.f2 = nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 2))

        # Quantum branches (TorchLayer wrappers)

```

```

self.c1 = qml.qnn.TorchLayer(circuit_1, weight_shapes_1)
self.c2 = qml.qnn.TorchLayer(circuit_2, weight_shapes_2)
self.c3 = qml.qnn.TorchLayer(circuit_3, weight_shapes_3)

# Final classical head
self.m1 = nn.Linear(2, 1)

def forward(self, d1, d2):
    b1 = self.c1(d1)                    # branch 1: direct quantum
    b2 = self.c2(self.f2(d2))           # branch 2: classical → quantum
    joined = torch.cat([b1, b2], dim=-1) # join outputs
    c3_out = self.c3(joined)
    return self.m1(c3_out.float())

```

3.4 Training Loop

Once the model is an `nn.Module`, training is standard PyTorch:

```

model      = SimpleHybrid()
optimizer  = torch.optim.Adam(model.parameters(), lr=0.01)
loss_fn    = nn.MSELoss()

for epoch in range(200):
    optimizer.zero_grad()
    predictions = model(X_train)
    loss = loss_fn(predictions, y_train)
    loss.backward()      # gradients flow through quantum + classical layers
    optimizer.step()

```

i Note

`loss.backward()` triggers PennyLane's differentiation for the quantum layers and PyTorch's autograd for the classical layers. No special handling is needed.

3.5 Weight Shape Reference

Ansatz	Shape
BasicEntanglerLayers	(n_layers, n_qubits)
StronglyEntanglingLayers	(n_layers, n_qubits, 3)
Single RY per wire	(n_qubits,)

3.6 Key Considerations

Interface consistency. The `QNode` must use `interface="torch"`. Passing a NumPy array into a `TorchLayer` will cause a type error.

Output shape. `qml.expval(qml.PauliZ(0))` returns a scalar per sample; the `TorchLayer` adds a batch dimension, giving shape `(batch,)`. Use `.unsqueeze(-1)` or `nn.Linear(1, ...)` to connect to subsequent layers.

Gradient method. `diff_method="adjoint"` is fastest but requires circuits without mid-circuit measurements or data reuploading. Fall back to `"parameter-shift"` for those cases.

3.7 Exercises

1. Build a hybrid model with one classical `nn.Linear(4, 2)` layer feeding into a 2-qubit quantum circuit, followed by `nn.Linear(1, 1)`. Print the model's parameter count.
2. Replace `StronglyEntanglingLayers` with `BasicEntanglerLayers`. Update the `weight_shapes` dictionary accordingly and verify training still works.
3. In the multi-branch architecture, swap the quantum branch `c1` for a classical `nn.Linear`. Does training speed change significantly?
4. Modify the training loop to record train and validation loss at each epoch, then plot the learning curves.

Part II

Part II: Practical Models

Chapter 4

Your First Quantum Regression Model

This chapter builds a complete variational quantum circuit that learns to approximate $\sin(x)$. Along the way we compare three ansatz architectures and introduce the concept of data reuploading.

4.1 Problem Setup

We want to learn a function $f : \mathbb{R} \rightarrow \mathbb{R}$ from noisy samples:

$$y_i = \sin(x_i) + \varepsilon_i, \quad x_i \in [0, 2\pi]$$

```
import numpy as np
import pennylane as qml

# Dataset
n_samples = 50
X = np.linspace(0, 2 * np.pi, n_samples)
y = np.sin(X)
```

4.2 Anatomy of a Variational QML Model

Every variational model has three components:

1. **Feature map** — encodes input x into the circuit
2. **Ansatz** — parameterised unitary $U(\theta)$
3. **Measurement** — extracts a scalar prediction

The model output is the expectation value of a Pauli-Z observable:

$$\hat{y}(x; \theta) = \langle 0 | U^\dagger(\theta) \phi^\dagger(x) Z_0 \phi(x) U(\theta) | 0 \rangle$$

4.3 Cost vs Loss

Throughout this book we distinguish two related quantities:

- **Loss** — the error on a single sample: $\ell_i = (\hat{y}_i - y_i)^2$
- **Cost** — the aggregate error over the training set: $\mathcal{C}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_i$

Optimisation minimises the cost.

4.4 Model 1 — Basic Entangler (Simplest)

```
n_qubits = 2
n_layers = 3
dev = qml.device("lightning.qubit", wires=n_qubits)

@qml.qnode(dev, diff_method='adjoint')
def qnn_model_1(params, x):
    # Feature map: encode scalar x onto qubit 0
    qml.RY(x, wires=0)

    # Ansatz: BasicEntanglerLayers
    # params shape: (n_layers, n_qubits)
    qml.BasicEntanglerLayers(params, wires=range(n_qubits), rotation=qml.RY)

    # Measurement
    return qml.expval(qml.PauliZ(0))
```

`BasicEntanglerLayers` applies one rotation per wire per layer, then a ring of CNOT gates. It requires `(n_layers, n_qubits)` parameters.

4.4.1 Training

```
params = np.random.uniform(0, np.pi, size=(n_layers, n_qubits))
optimizer = qml.AdamOptimizer(stepsize=0.05)

def cost(params):
    predictions = np.array([qnn_model_1(params, x) for x in X])
    return np.mean((predictions - y) ** 2)

for step in range(200):
    params, c = optimizer.step_and_cost(cost, params)
    if step % 50 == 0:
        print(f"Step {step:3d} | Cost = {c:.4f}")
```

4.5 Model 2 — Strong Entangler (More Expressive)

`StronglyEntanglingLayers` adds three rotation angles per qubit per layer (RZ-RY-RZ decomposition) plus a more flexible entanglement pattern:


```
@qml.qnode(dev, diff_method='adjoint')
def qnn_model_2(params, x):
    # Feature map: encode x onto all qubits
    for wire in range(n_qubits):
        qml.RY(x, wires=wire)

    # Ansatz: StronglyEntanglingLayers
    # params shape: (n_layers, n_qubits, 3)
    qml.StronglyEntanglingLayers(params, wires=range(n_qubits))

    return qml.expval(qml.PauliZ(0))

params_2 = np.random.uniform(0, np.pi, size=(n_layers, n_qubits, 3))
```

The additional expressibility often improves fit quality, but increases parameter count by a factor of 3.

4.6 Model 3 — Data Reuploading (Most Expressive)

Encoding data only once limits the model to functions that a single feature-map layer can represent. Reuploading breaks this limitation by re-injecting the input between ansatz layers:

```
@qml.qnode(dev, diff_method='parameter-shift') # adjoint incompatible with reuploading
def qnn_model_3(params, x):
    n_layers_local = params.shape[0] // n_qubits
    for l in range(n_layers_local):
        # Re-encode at each layer
        for wire in range(n_qubits):
            qml.RY(x, wires=wire)
        # One layer of entangler
        qml.BasicEntanglerLayers(
            params[l * n_qubits:(l + 1) * n_qubits].reshape(1, n_qubits),
            wires=range(n_qubits),
            rotation=qml.RY
        )
    return qml.expval(qml.PauliZ(0))
```

Warning

`diff_method='adjoint'` is not compatible with mid-circuit encoding layers. Use `'parameter-shift'` instead. Training will be slower (two circuit evaluations per parameter per step).

4.7 Optimiser Comparison

PennyLane provides several native optimisers:

Optimiser	Key property
<code>GradientDescentOptimizer</code>	Vanilla SGD; simple baseline
<code>AdamOptimizer</code>	Adaptive learning rates; usually best default
<code>RMSPropOptimizer</code>	Good for non-stationary objectives
<code>NesterovMomentumOptimizer</code>	Look-ahead momentum

```
opt = qml.AdamOptimizer(stepsize=0.05)
opt = qml.NesterovMomentumOptimizer(stepsize=0.01, momentum=0.9)
```

4.8 Initialisation Matters

The initial parameters strongly influence convergence:

```
# Large random initialisation - risk of barren plateaus for deep circuits
params = np.random.uniform(0, np.pi, ...)

# Small initialisation - avoids large initial gradients
params = np.random.uniform(0, 0.1, ...)

# Near-zero - starts close to identity
params = np.random.uniform(0, 0.01, ...)
```

For shallow circuits (1-2 layers, 2-4 qubits), random uniform $[0, \pi]$ works well.

4.9 Evaluating Fit Quality

```
y_pred = np.array([qnn_model_1(params, x) for x in X])
mse      = np.mean((y_pred - y) ** 2)
print(f"MSE = {mse:.4f}")

import matplotlib.pyplot as plt
plt.plot(X, y, label='sin(x)', linewidth=2)
plt.plot(X, y_pred, label='QNN fit', linestyle='--')
plt.legend(); plt.xlabel('x'); plt.ylabel('y')
plt.title('Quantum Regression')
plt.tight_layout()
plt.show()
```

4.10 Exercises

1. Train Model 1 with a single qubit (`n_qubits=1`) and a single layer (`n_layers=1`). Does it converge? Explain why or why not in terms of the model's expressibility.
2. Change the feature encoding from `qml.RY` to `qml.RX` in Model 1. Does the fit quality change for $\sin(x)$? Why might one rotation axis work better than another for a particular function?

3. Fit the harder dataset $y = \sin(0.7x + 0.5\pi)$ using Models 1 and 3. Which model achieves lower MSE? How many training steps does each need?
4. Plot the training cost curve for all three models on the same axes. Which model converges fastest? Which achieves the lowest final cost?
5. Use `qml.draw_mpl` to visualise all three circuits. Count the trainable parameters in each. What is the parameter-to-MSE trade-off?

Chapter 5

Quantum Estimator for Real Datasets

In Chapter 4 we fitted a known function. Here we build a *Quantum Estimator* — a quantum regression model — and apply it to real-world datasets. This chapter introduces proper train/test evaluation, the R^2 metric, and residuals analysis.

5.1 What Is a Quantum Estimator?

The term *estimator* (borrowed from scikit-learn) refers to a model that *estimates* a continuous target variable. Our quantum estimator is a variational circuit trained to minimise MSE on labelled data.

5.2 Datasets

We use three datasets of increasing complexity:

1. **Synthetic regression** (scikit-learn `make_regression`):

```
from sklearn.datasets import make_regression
X, y = make_regression(n_samples=100, n_features=1, noise=10, random_state=42)
```

2. **Real Estate (Taiwan)** — predict house price from location and age.

3. **Swiss Roll** — 3D manifold; tests whether the circuit can learn nonlinear structure from 2D projections.

```
from sklearn.datasets import make_swiss_roll
X_3d, t = make_swiss_roll(n_samples=200, noise=0.1, random_state=42)
X = X_3d[:, [0, 2]] # project to 2D
y = t               # colour coordinate as target
```

5.3 Preprocessing

For angle encoding, features must lie in a range the circuit can distinguish. Standard scaling followed by a per-feature margin:

```

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)      # zero mean, unit variance

x_angle_margin = 0.3                    # compress to [-0.3, 0.3]
# Usage inside circuit: qml.RY(x * np.pi * x_angle_margin, wires=i)

```

A margin of 0.3 keeps inputs away from the poles of the Bloch sphere. You can increase it to 0.5 for more expressive encoding, but watch for vanishing gradients.

5.4 The Estimator Circuit

```

import numpy as np
import pennylane as qml

n_qubits      = 4
n_layers      = 3
x_angle_margin = 0.3
dev = qml.device("lightning.qubit", wires=n_qubits)

@qml.qnode(dev, diff_method='adjoint')
def qnn_estimator(params, x):
    # Feature map: encode each (scaled) feature onto one qubit
    for i, xi in enumerate(x):
        qml.RY(xi * np.pi * x_angle_margin, wires=i)

    # Ansatz: choose one
    qml.BasicEntanglerLayers(params, wires=range(n_qubits), rotation=qml.RY)
    # -- or --
    # qml.StronglyEntanglingLayers(params, wires=range(n_qubits))

    # Measurement: scalar prediction from qubit 0
    return qml.expval(qml.PauliZ(0))

```

For `BasicEntanglerLayers`: `params.shape = (n_layers, n_qubits)`. For `StronglyEntanglingLayers`: `params.shape = (n_layers, n_qubits, 3)`.

5.5 Train / Test Split

```

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

```

5.6 Training

```

params = np.random.uniform(0, np.pi, size=(n_layers, n_qubits))
optimizer = qml.AdamOptimizer(stepsize=0.05)

def cost(params, X_batch, y_batch):
    preds = np.array([qnn_estimator(params, x) for x in X_batch])
    return np.mean((preds - y_batch) ** 2)

n_epochs = 200
for epoch in range(n_epochs):
    params, train_loss = optimizer.step_and_cost(
        lambda p: cost(p, X_train, y_train), params
    )
    if epoch % 50 == 0:
        print(f"Epoch {epoch:3d} | Train MSE = {train_loss:.4f}")

```

5.7 Evaluation Metrics

5.7.1 Mean Squared Error

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Lower is better; same units as y^2 .

5.7.2 R^2 (Coefficient of Determination)

$$R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

$R^2 = 1$ is a perfect fit; $R^2 = 0$ means the model does no better than predicting the mean; $R^2 < 0$ is worse than predicting the mean.

```

from sklearn.metrics import mean_squared_error, r2_score

y_pred_train = np.array([qnn_estimator(params, x) for x in X_train])
y_pred_test  = np.array([qnn_estimator(params, x) for x in X_test])

print(f"Train MSE = {mean_squared_error(y_train, y_pred_train):.4f}")
print(f"Test  MSE = {mean_squared_error(y_test, y_pred_test):.4f}")
print(f"Train R2  = {r2_score(y_train, y_pred_train):.4f}")
print(f"Test  R2  = {r2_score(y_test, y_pred_test):.4f}")

```

5.8 Residuals Analysis

Residuals are the signed prediction errors:

$$r_i = y_i - \hat{y}_i$$

Plotting residuals reveals systematic bias:

```
import matplotlib.pyplot as plt

residuals = y_test - y_pred_test

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Residuals vs predicted
axes[0].scatter(y_pred_test, residuals, alpha=0.6)
axes[0].axhline(0, color='red', linewidth=1)
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Residual (y - ŷ)')
axes[0].set_title('Residuals vs Fitted')

# Residual distribution
axes[1].hist(residuals, bins=20, edgecolor='black')
axes[1].set_xlabel('Residual')
axes[1].set_ylabel('Count')
axes[1].set_title('Residual Distribution')

plt.tight_layout()
plt.show()
```

A good model shows residuals that are: - Centred near zero (no systematic bias) - Roughly symmetric - Without obvious structure (no fan-shape or curve)

5.9 Ansatz Comparison

Ansatz	Params (4q, 3l)	Typical test R ²
BasicEntanglerLayers	12	Moderate
StronglyEntanglingLayers	36	Higher
Basic + reuploading	12 × layers	Highest (slower)

More parameters help — up to a point. On small datasets, overfitting is a risk even for quantum models.

5.10 Exercises

1. Train the estimator on `make_regression` with 1 and 4 features. How does increasing the feature count affect test R²? What circuit change is needed?
2. Vary `x_angle_margin` from 0.1 to 0.9 in steps of 0.2. Plot test MSE vs margin. Where is the sweet spot?

3. Switch from `BasicEntanglerLayers` to `StronglyEntanglingLayers`. Does test R^2 improve? How does training time change?
4. Add data reuploading (with `diff_method='parameter-shift'`). Compare test MSE to the single-encoding version after 200 epochs.
5. Plot the residuals for train and test sets on the same figure. Is there evidence of overfitting or underfitting?

Chapter 6

Quantum Classifier

This chapter builds a binary quantum classifier using PyTorch and PennyLane together, applied to the UCI Automobile dataset. The same dataset is used in the next chapter for a classical baseline, enabling a head-to-head comparison.

6.1 Dataset: Automobile Insurance Risk

The UCI Automobile dataset contains technical specifications of cars alongside a symbolic risk rating (`symboling`) that reflects insurance risk. We convert this into a binary target:

Risk	Label
≤ 0 (safe)	0
> 0 (risky)	1

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

df = pd.read_csv('../data/Automobile.csv')

# Keep numeric columns, drop rows with missing values
df_num = df.select_dtypes(include=[np.number]).dropna()

# Binary target
y = (df_num['symboling'] > 0).astype(float).values

# Features: all numeric columns except target
X_raw = df_num.drop(columns=['symboling']).values
```

6.2 Dimensionality Reduction

The dataset has ~13 numeric features but our quantum circuit supports only `n_qubits` features. We use PCA to reduce:

```
from sklearn.decomposition import PCA

n_qubits = 4
pca = PCA(n_components=n_qubits, random_state=42)
X_pca = pca.fit_transform(X_raw)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_pca)
```

Alternatives to PCA:

- **SelectKBest with k^2** — selects the k most informative features (requires non-negative inputs after shifting)
- **Manual selection** — domain knowledge about which car specs predict risk

6.3 The Quantum Circuit

```
import pennylane as qml
import torch
import torch.nn as nn

dev = qml.device("default.qubit", wires=n_qubits)
n_layers = 3

@qml.qnode(dev, interface="torch", diff_method="adjoint")
def quantum_circuit(inputs, weights):
    # inputs: shape (n_qubits,) - one feature per qubit
    # weights: shape (n_layers, n_qubits, 3)
    qml.AngleEmbedding(inputs, wires=range(n_qubits), rotation='Y')
    qml.StronglyEntanglingLayers(weights, wires=range(n_qubits))
    return qml.expval(qml.PauliZ(0)) # scalar in [-1, +1]
```

6.4 Quantum Classifier Model

```
class QuantumClassifier(nn.Module):
    def __init__(self, n_qubits, n_layers):
        super().__init__()

        # Quantum layer
        weight_shapes = {"weights": (n_layers, n_qubits, 3)}
        self.quantum = qml.qnn.TorchLayer(quantum_circuit, weight_shapes)
```

```

    # Classical head: maps quantum scalar → logit
    self.classical = nn.Sequential(
        nn.Linear(1, 64),
        nn.ReLU(),
        nn.Linear(64, 1)
    )

    def forward(self, x):
        # x: shape (batch, n_qubits)
        q_out = self.quantum(x).float().unsqueeze(-1) # (batch, 1)
        return self.classical(q_out)                 # (batch, 1) - logit

```

The quantum layer outputs a scalar per sample; the classical head maps it to a binary logit. Using `BCEWithLogitsLoss` (which applies sigmoid internally) is numerically more stable than applying sigmoid manually.

6.5 Training

```

# Torch tensors
X_tensor = torch.tensor(X_scaled, dtype=torch.float32)
y_tensor = torch.tensor(y, dtype=torch.float32).unsqueeze(-1)

# Train/test split
from sklearn.model_selection import train_test_split
idx = list(range(len(X_tensor)))
train_idx, test_idx = train_test_split(idx, test_size=0.2, random_state=42)

X_train = X_tensor[train_idx]; y_train = y_tensor[train_idx]
X_test = X_tensor[test_idx]; y_test = y_tensor[test_idx]

# Model
model = QuantumClassifier(n_qubits, n_layers)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
loss_fn = nn.BCEWithLogitsLoss()

# Training loop
for epoch in range(100):
    model.train()
    optimizer.zero_grad()
    logits = model(X_train)
    loss = loss_fn(logits, y_train)
    loss.backward()
    optimizer.step()

    if epoch % 20 == 0:
        model.eval()
        with torch.no_grad():

```

```

        val_logits = model(X_test)
        val_loss    = loss_fn(val_logits, y_test)
        print(f"Epoch {epoch:3d} | Train={loss.item():.4f} | Val={val_loss.item():.4f}")

```

6.6 Evaluation

```

model.eval()
with torch.no_grad():
    test_logits = model(X_test)
    probs = torch.sigmoid(test_logits)
    preds = (probs > 0.5).float()

from sklearn.metrics import accuracy_score, classification_report

acc = accuracy_score(y_test.numpy(), preds.numpy())
print(f"Test accuracy: {acc:.4f}")
print(classification_report(y_test.numpy(), preds.numpy(),
                           target_names=['Safe', 'Risky']))

```

6.7 Making Predictions on New Data

```

fantasy_car = np.array([
    2500, # curb-weight (kg)
    190,  # horsepower
    4.5,  # length (m)
    120,  # city-mpg
])
x_new = scaler.transform(pca.transform([fantasy_car]))
x_tensor = torch.tensor(x_new, dtype=torch.float32)

model.eval()
with torch.no_grad():
    prob = torch.sigmoid(model(x_tensor)).item()
print(f"Risk probability: {prob:.2%}")

```

6.8 Key Design Choices

Dimensionality reduction strategy. PCA captures maximum variance but may discard class-discriminative information. Chi-squared feature selection preserves interpretability. Neither is universally superior.

Data shuffling. Always shuffle before splitting. The Automobile dataset is ordered by risk category, so a non-shuffled split creates biased train/test sets.

Quantum vs classical head. The classical `nn.Sequential` head after the quantum layer is essential: `expval` outputs are in $[-1, +1]$ with non-linear quantum features, and the linear head

transforms this into a calibrated logit.

6.9 Exercises

1. Replace PCA with `SelectKBest(chi2, k=n_qubits)`. Shift features to be non-negative first. Does accuracy improve?
2. Remove the classical head (directly apply sigmoid to the quantum output). How does accuracy change? What does this tell you about the role of the head?
3. Increase `n_qubits` to 6 and `n_layers` to 4. Does test accuracy improve? What happens to training time?
4. Plot the ROC curve for the quantum classifier. What is the AUC?
5. Retrain the model 5 times with different random seeds. Report mean and standard deviation of test accuracy. How stable is quantum training?

Chapter 7

Classical Baseline Classifier

Before celebrating (or dismissing) quantum classifier results, we need a fair classical baseline trained on the same data. This chapter builds a comparable PyTorch-only model and sets up a rigorous comparison framework.

7.1 Why a Baseline Matters

Quantum classifiers are not necessarily better than classical ones — especially on small, structured datasets where classical deep learning has decades of optimisation advantage. A baseline answers the key question: **does the quantum model add value?**

A meaningful comparison requires: - Same dataset and preprocessing - Same train/test split (same random seed) - Same number of training epochs - Same optimiser and learning rate - Roughly comparable parameter count

7.2 Classical Classifier Architecture

```
import torch
import torch.nn as nn

class ClassicalClassifier(nn.Module):
    def __init__(self, input_dim: int):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(input_dim, 128),
            nn.ReLU(),
            nn.Dropout(p=0.2),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 1)          # logit output; sigmoid in loss
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
return self.model(x)
```

Parameter count: $4 \times 128 + 128 + 128 \times 64 + 64 + 64 \times 1 + 1 = 8897$ for `input_dim=4`. This is far more than the quantum classifier's ~ 50 parameters, so the classical model has a structural advantage.

7.3 Training (Same Setup as Quantum)

```
# Reuse X_train, X_test, y_train, y_test from Chapter 6
# Here we feed the PCA-reduced 4-feature data directly (no quantum layer)

model_cl = ClassicalClassifier(input_dim=4)
optimizer = torch.optim.Adam(model_cl.parameters(), lr=0.01)
loss_fn = nn.BCEWithLogitsLoss()

train_losses, val_losses = [], []

for epoch in range(100):
    model_cl.train()
    optimizer.zero_grad()
    logits = model_cl(X_train)
    loss = loss_fn(logits, y_train)
    loss.backward()
    optimizer.step()
    train_losses.append(loss.item())

    model_cl.eval()
    with torch.no_grad():
        val_logits = model_cl(X_test)
        val_loss = loss_fn(val_logits, y_test)
    val_losses.append(val_loss.item())
```

7.4 Evaluation

```
from sklearn.metrics import (
    accuracy_score, classification_report,
    roc_auc_score, roc_curve
)
import matplotlib.pyplot as plt

model_cl.eval()
with torch.no_grad():
    cl_probs = torch.sigmoid(model_cl(X_test)).numpy().ravel()
    cl_preds = (cl_probs > 0.5).astype(float)

acc_cl = accuracy_score(y_test.numpy(), cl_preds)
```

```

auc_cl = roc_auc_score(y_test.numpy(), cl_probs)

print(f"Classical | Acc = {acc_cl:.4f} | AUC = {auc_cl:.4f}")
print(classification_report(y_test.numpy(), cl_preds,
                           target_names=['Safe', 'Risky']))

```

7.5 Side-by-Side Comparison

```

# Assume quantum model results stored as acc_qc, auc_qc, qc_probs

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Learning curves
axes[0].plot(train_losses, label='Classical train', color='blue')
axes[0].plot(val_losses, label='Classical val', color='blue', linestyle='--')
# add quantum curves analogously
axes[0].set_xlabel('Epoch'); axes[0].set_ylabel('BCE Loss')
axes[0].set_title('Learning Curves'); axes[0].legend()

# ROC curves
fpr_cl, tpr_cl, _ = roc_curve(y_test.numpy(), cl_probs)
axes[1].plot(fpr_cl, tpr_cl, label=f'Classical (AUC={auc_cl:.3f})', color='blue')
axes[1].plot([0, 1], [0, 1], 'k--', linewidth=0.8)
axes[1].set_xlabel('FPR'); axes[1].set_ylabel('TPR')
axes[1].set_title('ROC Curve'); axes[1].legend()

plt.tight_layout(); plt.show()

```

7.6 Interpreting the Results

On the Automobile dataset (small: ~200 samples after cleaning), typical outcomes:

Model	Accuracy	AUC	Training time
Classical (4 features in)	~70–80%	~0.75–0.85	Fast
Quantum (4 qubits)	~65–78%	~0.70–0.82	Slow

Quantum classifiers often lag behind on small structured datasets because:

1. **Limited data** — quantum models have fewer parameters to exploit structure
2. **Optimiser maturity** — classical Adam is highly tuned; quantum landscapes have barren plateaus
3. **Simulation overhead** — the speedup of quantum hardware is not available in simulation

This does not mean quantum classifiers are useless — their advantage may appear on data with genuine quantum structure (e.g., quantum chemistry, quantum sensor data).

7.7 When Might Quantum Win?

Theoretical arguments suggest quantum advantage in classification when:

- The data is generated by a quantum process
- The feature Hilbert space is exponentially large and classical models cannot efficiently approximate the kernel
- The circuit depth required to express the classifier is fundamentally lower than for any classical circuit

For classical structured tabular data like Automobile, no such advantage has been demonstrated.

7.8 Exercises

1. Train the classical model with `input_dim=13` (all numeric features, no PCA). Compare accuracy to the PCA-4 version. Does removing the dimensionality reduction step help the classical model more than the quantum model?
2. Add `nn.BatchNorm1d` after each `nn.ReLU` in the classical model. Does convergence improve?
3. Build a *fair* comparison by constraining the classical model to the same parameter count as the quantum classifier (~50 parameters). What architecture achieves this? How does accuracy compare?
4. Run both models 10 times with different random seeds. Report mean $\pm$ std for accuracy and AUC. Which model is more stable?
5. Try the quantum model with `BasicEntanglerLayers` instead of `StronglyEntanglingLayers`. Does the gap vs the classical baseline change?

Part III

Part III: Advanced Topics

Chapter 8

Hybrid Quantum-Classical Logistic Regression

Logistic regression is the canonical binary classifier: it outputs a probability $\hat{p} \in [0, 1]$ via a sigmoid transformation of a linear score. In this chapter we replace the linear score with a variational quantum circuit, producing a *quantum logistic regression* model.

8.1 From Linear to Quantum Score

Classical logistic regression:

$$\hat{p} = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Quantum logistic regression:

$$\hat{p} = \sigma(\langle Z_0 \rangle_\theta(\mathbf{x}))$$

where $\langle Z_0 \rangle_\theta(\mathbf{x})$ is the PauliZ expectation value of the circuit parameterised by θ with input $\mathbf{x}$.

8.2 Binary Cross-Entropy Loss

Logistic regression minimises the **Binary Cross-Entropy (BCE)** loss:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

Compared to MSE, BCE is: - Convex in the sigmoid output (favourable optimisation landscape)
- Properly calibrated for probability outputs - Heavily penalising confident wrong predictions

8.3 The Quantum Circuit

```
import pennylane as qml
import torch
import torch.nn as nn

n_qubits = 4
n_layers = 3
dev = qml.device("lightning.qubit", wires=n_qubits)

@qml.qnode(dev, interface="torch", diff_method="adjoint")
def quantum_circuit(inputs, weights):
    # inputs: (n_qubits,) - pre-normalised features
    # weights: (n_layers, n_qubits, 3)
    qml.AngleEmbedding(inputs, wires=range(n_qubits), rotation='Y')
    qml.StronglyEntanglingLayers(weights, wires=range(n_qubits))
    return qml.expval(qml.PauliZ(0)) # [-1, +1]
```

8.4 Model: expval Path

The simplest design applies sigmoid directly to the quantum output:

```
class QuantumLogisticRegression(nn.Module):
    def __init__(self, n_qubits, n_layers):
        super().__init__()
        weight_shapes = {"weights": (n_layers, n_qubits, 3)}
        self.quantum = qml.qnn.TorchLayer(quantum_circuit, weight_shapes)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # x: (batch, n_qubits)
        q_out = self.quantum(x).float() # (batch,) [-1, +1]
        return self.sigmoid(q_out).unsqueeze(-1) # (batch, 1) [0, 1]
```

Use BCELoss (not BCEWithLogitsLoss) when sigmoid is applied explicitly:

```
loss_fn = nn.BCELoss()
```

8.5 Model: probs Path (Alternative)

Instead of mapping $[-1, +1]$ through sigmoid, we can directly read the probability of measuring $|1\rangle$:

```
@qml.qnode(dev, interface="torch", diff_method="adjoint")
def circuit_probs(inputs, weights):
    qml.AngleEmbedding(inputs, wires=range(n_qubits), rotation='Y')
    qml.StronglyEntanglingLayers(weights, wires=range(n_qubits))
    return qml.probs(wires=0) # [P(|0>), P(|1>)] - already in [0, 1]
```

```

class QuantumLogisticProbs(nn.Module):
    def __init__(self, n_qubits, n_layers):
        super().__init__()
        weight_shapes = {"weights": (n_layers, n_qubits, 3)}
        self.quantum = qml.qnn.TorchLayer(circuit_probs, weight_shapes)

    def forward(self, x):
        probs = self.quantum(x)          # (batch, 2)
        return probs[:, 1].unsqueeze(-1) # P(|1): (batch, 1)

```

This avoids the sigmoid entirely — the model directly outputs a quantum probability.

i Note

The `probs` path is conceptually cleaner but may have a flatter training landscape because probabilities are bounded in $[0, 1]$ from the start. The `expval + sigmoid` path gives the optimiser more signal in the early training phase.

8.6 Training

```

model      = QuantumLogisticRegression(n_qubits, n_layers)
optimizer  = torch.optim.Adam(model.parameters(), lr=0.01)
loss_fn    = nn.BCELoss()

for epoch in range(150):
    model.train()
    optimizer.zero_grad()
    p_hat = model(X_train)
    loss = loss_fn(p_hat, y_train)
    loss.backward()
    optimizer.step()

```

8.7 Adding a Classical Booster Layer

When the quantum circuit alone struggles, a small classical layer can help:

```

class HybridLogistic(nn.Module):
    def __init__(self, n_qubits, n_layers, use_booster=False):
        super().__init__()
        weight_shapes = {"weights": (n_layers, n_qubits, 3)}
        self.quantum = qml.qnn.TorchLayer(quantum_circuit, weight_shapes)
        self.use_booster = use_booster

    if use_booster:
        # Thin classical booster
        self.booster = nn.Sequential(

```

```

        nn.Linear(1, 32),
        nn.ReLU(),
        nn.Linear(32, 1),
        nn.Sigmoid()
    )
    else:
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        q = self.quantum(x).float().unsqueeze(-1)  # (batch, 1)
        if self.use_booster:
            return self.booster(q)
        return self.sigmoid(q)

```

The booster adds 65 parameters and can substantially improve accuracy when the raw quantum score is not linearly separable from the target.

8.8 LBFGS Optimiser (Advanced)

L-BFGS is a quasi-Newton second-order method that can converge much faster than Adam on low-parameter problems. PyTorch requires a closure:

```

optimizer_lbfgs = torch.optim.LBFGS(
    model.parameters(),
    lr=0.5,
    max_iter=20,
    history_size=10
)

def closure():
    optimizer_lbfgs.zero_grad()
    p_hat = model(X_train)
    loss = loss_fn(p_hat, y_train)
    loss.backward()
    return loss

for epoch in range(50):
    loss = optimizer_lbfgs.step(closure)
    print(f"Epoch {epoch:2d} | Loss = {loss.item():.4f}")

```

L-BFGS typically converges in 20–50 iterations vs 100–200 for Adam, but each iteration is more expensive.

8.9 Exercises

1. Train the `expval` model and the `probs` model on the Automobile dataset. Compare test accuracy and AUC. Which converges faster?

2. Enable `use_booster=True`. Does the booster help more on the `expval` or `probs` path?
3. Replace `StronglyEntanglingLayers` with `BasicEntanglerLayers`. Does BCE loss still converge? Does accuracy change?
4. Switch from Adam to LBFGS. How many epochs does LBFGS need to reach the same loss that Adam achieves in 100 epochs?
5. Plot the predicted probability $\hat{p}$ against the true label for the test set. Is the model well-calibrated (are samples with $\hat{p} \approx 0.7$ correct about 70% of the time)?

Chapter 9

Quantum Autoencoder: Pure PennyLane

A *Quantum Autoencoder* (QAE) compresses a quantum state to fewer qubits and reconstructs it, analogous to a classical autoencoder but operating in Hilbert space. This chapter implements a QAE for time-series denoising using only PennyLane.

9.1 The Problem: Denoising Time Series

We use the **Mackey-Glass equation** — a nonlinear delay-differential equation widely used to benchmark time-series models:

$$\frac{dx(t)}{dt} = \frac{\beta x(t - \tau)}{1 + x(t - \tau)^n} - \gamma x(t)$$

with typical parameters $\beta = 0.2$, $\tau = 17$, $n = 10$, $\gamma = 0.1$. The system exhibits complex quasi-chaotic dynamics for large τ .

```
# Mackey-Glass generator (simplified Euler integration)
def mackey_glass(n_samples, tau=17, beta=0.2, gamma=0.1, n=10, dt=1.0, seed=42):
    np.random.seed(seed)
    history = np.ones(tau + 1) * 0.9
    x = list(history)
    for _ in range(n_samples):
        x_tau = x[-tau]
        dx = beta * x_tau / (1 + x_tau**n) - gamma * x[-1]
        x.append(x[-1] + dt * dx)
    return np.array(x[tau:])

ts_clean = mackey_glass(n_samples=1000)
```

Noise is added for denoising experiments:

```
def add_noise(ts, noise_level=0.05):
    return ts + np.random.normal(0, noise_level * ts.std(), size=ts.shape)
```



```
ts_noisy = add_noise(ts_clean, noise_level=0.1)
```

9.2 Sliding Windows

Raw time series must be segmented into fixed-length windows for batch processing:

```
def sliding_windows(ts, window_size):
    """Returns array of shape (n_windows, window_size)."""
    return np.array([ts[i:i+window_size] for i in range(len(ts) - window_size)])

window_size = 4          # matches n_qubits (angle encoding)
X_windows    = sliding_windows(ts_noisy, window_size)    # (n_windows, 4)
y_windows    = sliding_windows(ts_clean, window_size)    # (n_windows, 4)
```

9.3 QAE Architecture

A QAE has three stages:

Input state $|\text{_input}\rangle$ (n qubits)

Encoder $n_{\text{qubits}} \rightarrow n_{\text{latent}}$ qubits

Latent qubits	Trash qubits
(preserved)	(measured, reset to $ 0\rangle$)

Decoder $n_{\text{latent}} \rightarrow n_{\text{qubits}}$ (mirror of encoder)

Reconstructed state $|\text{_output}\rangle$

The loss measures the distance between the input and output states. The ideal loss is the **trash qubit fidelity**: if all information is captured in the latent space, the trash qubits should be in $|0\rangle$ after the encoder.

9.4 SWAP Test for State Comparison

The **SWAP test** computes the inner product $|\langle\psi|\phi\rangle|^2$ without collapsing either state:

Ancilla	$ 0\rangle$	H	CSWAP	H	measure
State 1	$ \psi\rangle$				
State 2	$ \phi\rangle$				

$$P(\text{ancilla} = |0\rangle) = (1 + |\langle 0 | \psi \rangle|^2) / 2$$

$$\text{Fidelity} = |\langle \psi | \phi \rangle|^2 = 2P(|0\rangle_{\text{ancilla}}) - 1$$

```

n_qubits = 4
n_latent = 2 # compressed representation
n_trash = n_qubits - n_latent # = 2
dev_swap = qml.device("default.qubit", wires=2 * n_qubits + 1) # +1 ancilla

@qml.qnode(dev_swap)
def swap_test(state_1, state_2):
    """Measure fidelity between two n-qubit states."""
    # Ancilla qubit: index 0
    qml.Hadamard(wires=0)
    # Prepare state_1 on qubits 1..n_qubits
    qml.AmplitudeEmbedding(state_1, wires=range(1, n_qubits + 1), normalize=True)
    # Prepare state_2 on qubits n_qubits+1..2*n_qubits
    qml.AmplitudeEmbedding(state_2, wires=range(n_qubits + 1, 2*n_qubits + 1), normalize=True)
    # CSWAP controlled on ancilla
    for i in range(n_qubits):
        qml.CSWAP(wires=[0, i + 1, n_qubits + i + 1])
    qml.Hadamard(wires=0)
    return qml.probs(wires=0) # [P(|0>), P(|1>)]

def fidelity(s1, s2):
    probs = swap_test(s1, s2)
    return 2 * probs[0] - 1 # [0, 1]

```

9.5 QAE Circuit

```

dev_qae = qml.device("lightning.qubit", wires=n_qubits)

@qml.qnode(dev_qae, diff_method="adjoint")
def qae_circuit(params, x):
    """
    params: (n_layers, n_qubits, 3) - shared encoder + decoder
    x:      (n_qubits,) - normalised input window
    """
    # Data encoding
    qml.AngleEmbedding(x, wires=range(n_qubits), rotation='Y')

    # Encoder: compress to n_latent qubits
    qml.StronglyEntanglingLayers(params, wires=range(n_qubits))

    # Trash: reset n_trash qubits (simulate loss of information)

```

```

for wire in range(n_latent, n_qubits):
    qml.measure(wire)          # collapse → classical bit
    # In simulation: we measure but cannot reset; use adjoint trick instead

# Decoder: mirror (adjoint) of encoder
qml.adjoint(qml.StronglyEntanglingLayers)(params, wires=range(n_qubits))

return qml.expval(qml.PauliZ(0))

```

i Note

Resetting mid-circuit qubits requires hardware or the `lightning.qubit` plugin with dynamic circuits enabled. In practice, the loss function is designed to *incentivise* the trash qubits to end up near $|0\rangle$, which achieves the same effect.

9.6 Loss Function

We minimise the expected value of the trash qubit observable (encouraging $|0\rangle$):

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{j \in \text{trash}} \langle Z_j \rangle(\mathbf{x}_i, \theta)$$

$\langle Z_j \rangle = +1$ when qubit j is in $|0\rangle$, so minimising $-\langle Z_j \rangle$ pushes the trash qubits toward $|0\rangle$.

9.7 Training Loop

```

params = np.random.uniform(-np.pi, np.pi, size=(n_layers, n_qubits, 3))
optimizer = qml.AdamOptimizer(stepsize=0.02)

def cost(params, X_batch):
    losses = []
    for x in X_batch:
        # Measure trash qubits
        trash_expvals = [qae_circuit(params, x) for _ in range(n_trash)]
        losses.append(-np.mean(trash_expvals))
    return np.mean(losses)

n_epochs = 200
batch_size = 16

for epoch in range(n_epochs):
    batch_idx = np.random.choice(len(X_windows), batch_size, replace=False)
    X_batch = X_windows[batch_idx]
    params, c = optimizer.step_and_cost(lambda p: cost(p, X_batch), params)

```

```

if epoch % 50 == 0:
    print(f"Epoch {epoch:3d} | Cost = {c:.4f}")

```

9.8 Evaluation

```

# Reconstruct test windows and compare to clean signal
def reconstruct(params, X_noisy):
    reconstructed = []
    for x in X_noisy:
        # Forward pass through encoder+decoder
        x_norm = x / np.linalg.norm(x)
        output = qae_circuit(params, x_norm)
        reconstructed.append(output)
    return np.array(reconstructed)

X_test_windows = sliding_windows(ts_noisy[-200:], window_size)
y_test_windows = sliding_windows(ts_clean[-200:], window_size)

y_hat = reconstruct(params, X_test_windows)
mse_qae = np.mean((y_hat - y_test_windows[:, 0]) ** 2)
mse_raw = np.mean((X_test_windows[:, 0] - y_test_windows[:, 0]) ** 2)

print(f"Noisy input MSE: {mse_raw:.4f}")
print(f"QAE output MSE: {mse_qae:.4f}")
print(f"SNR improvement: {10 * np.log10(mse_raw / mse_qae):.1f} dB")

```

9.9 Exercises

1. Vary `n_latent` from 1 to `n_qubits - 1`. How does compression ratio affect reconstruction MSE?
2. Replace the Mackey-Glass series with a noisy sine wave. Does the QAE denoise it more or less effectively? Why might it differ?
3. Run 9 independent training instances with different random seeds. Report the distribution of final costs. How much variance is there in quantum training?
4. Compare training with `diff_method='adjoint'` vs `'parameter-shift'`. How much slower is parameter-shift? When would you prefer it?
5. Implement the SWAP test fidelity as the loss (instead of the trash qubit approach). Does it converge? Which loss is easier to optimise?

Chapter 10

Hybrid and Mirror QAE

This chapter extends the pure PennyLane QAE from Chapter 9 in two directions:

1. **Hybrid QAE** — wrapping the circuit as a PyTorch `nn.Module` and adding classical layers
2. **Mirror QAE** — a staged training strategy that trains decoder and encoder separately, preserving phase information

10.1 Hybrid QAE with PyTorch

Rewriting the QAE as a `nn.Module` gives access to the full PyTorch ecosystem: `DataLoader`, learning rate schedulers, early stopping, and classical layers.

10.1.1 Circuit as TorchLayer

```
import pennylane as qml
import torch
import torch.nn as nn

n_qubits = 4
n_latent = 2
n_layers = 3
dev = qml.device("lightning.qubit", wires=n_qubits)

@qml.qnode(dev, interface="torch", diff_method="adjoint")
def qae_torch(inputs, weights):
    # inputs: (n_qubits,)
    # weights: (n_layers, n_qubits, 3)
    qml.AngleEmbedding(inputs, wires=range(n_qubits), rotation='Y')
    qml.StronglyEntanglingLayers(weights, wires=range(n_qubits))
    # Return trash qubit expectation values
    return [qml.expval(qml.PauliZ(w)) for w in range(n_latent, n_qubits)]

weight_shapes = {"weights": (n_layers, n_qubits, 3)}
qae_layer = qml.qnn.TorchLayer(qae_torch, weight_shapes)
```

10.1.2 Hybrid Model

```

class HybridQAE(nn.Module):
    def __init__(self, n_qubits, n_latent):
        super().__init__()
        self.n_trash = n_qubits - n_latent

        # Classical pre-encoder: compress features before quantum layer
        self.pre = nn.Sequential(
            nn.Linear(n_qubits, n_qubits * 2),
            nn.Tanh(),
            nn.Linear(n_qubits * 2, n_qubits),
        )

        # Quantum core
        self.qae = qae_layer

        # Classical post-decoder: refine reconstruction
        self.post = nn.Sequential(
            nn.Linear(self.n_trash, n_qubits),
            nn.Tanh(),
            nn.Linear(n_qubits, n_qubits),
        )

    def forward(self, x):
        # x: (batch, n_qubits)
        x_pre = self.pre(x)                                # (batch, n_qubits)
        trash = self.qae(x_pre).float()                     # (batch, n_trash) - trash expvals
        recon = self.post(trash)                            # (batch, n_qubits)
        return recon, trash

```

The model returns both the reconstruction and the trash qubit values. The loss combines reconstruction error and trash qubit penalty:

$$\mathcal{L} = \lambda_1 \underbrace{\|\mathbf{x}_{\text{clean}} - \hat{\mathbf{x}}\|^2}_{\text{reconstruction}} - \lambda_2 \underbrace{\sum_j \langle Z_j \rangle_{\text{trash}}}_{\text{compression}}$$

```

def hybrid_qae_loss(recon, x_clean, trash_expvals, lam1=1.0, lam2=0.5):
    recon_loss = nn.functional.mse_loss(recon, x_clean)
    trash_loss = -trash_expvals.mean()          # push trash toward |0
    return lam1 * recon_loss + lam2 * trash_loss

```

10.2 Mirror QAE: Three-Stage Training

The Mirror QAE addresses a fundamental problem: if we train the full encoder+decoder jointly from scratch, the encoder may learn to store everything in the trash space and nothing in the latent space.

The solution is a structured three-stage protocol:

10.2.1 Stage 1 — Train the Decoder on Clean Data

The decoder (adjoint of the encoder circuit) first learns to *recognise* clean data. It is trained to output clean windows given clean inputs, minimising trash qubit energy.

```
@qml.qnode(dev, interface="torch", diff_method="adjoint")
def decoder_circuit(inputs, weights):
    """Decoder only: adjoint of the encoder."""
    qml.AngleEmbedding(inputs, wires=range(n_qubits), rotation='Y')
    # Adjoint (inverse) of the encoder ansatz
    qml.adjoint(qml.StronglyEntanglingLayers)(weights, wires=range(n_qubits))
    return [qml.expval(qml.PauliZ(w)) for w in range(n_qubits)]

decoder_layer = qml.qnn.TorchLayer(
    decoder_circuit, {"weights": (n_layers, n_qubits, 3)}
)

# Stage 1 training: minimise reconstruction error on CLEAN data
optimizer_s1 = torch.optim.Adam(decoder_layer.parameters(), lr=0.01)
for epoch in range(100):
    optimizer_s1.zero_grad()
    recon = decoder_layer(X_clean_train).float()
    loss = nn.functional.mse_loss(recon, X_clean_train)
    loss.backward()
    optimizer_s1.step()
```

10.2.2 Stage 2 — Train the Encoder on Noisy Data

With the decoder weights *frozen*, we train the encoder to compress noisy inputs into a latent representation that the fixed decoder can reconstruct cleanly.

```
@qml.qnode(dev, interface="torch", diff_method="adjoint")
def encoder_circuit(inputs, enc_weights):
    qml.AngleEmbedding(inputs, wires=range(n_qubits), rotation='Y')
    qml.StronglyEntanglingLayers(enc_weights, wires=range(n_qubits))
    return [qml.expval(qml.PauliZ(w)) for w in range(n_qubits)]

encoder_layer = qml.qnn.TorchLayer(
    encoder_circuit, {"enc_weights": (n_layers, n_qubits, 3)}
)

class MirrorQAE_Stage2(nn.Module):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
```

```

def forward(self, x_noisy):
    latent = self.encoder(x_noisy).float()    # encode noisy input
    recon = self.decoder(latent).float()      # decode with fixed decoder
    return recon

# Freeze decoder weights
for p in decoder_layer.parameters():
    p.requires_grad = False

model_s2 = MirrorQAE_Stage2(encoder_layer, decoder_layer)
opt_s2 = torch.optim.Adam(encoder_layer.parameters(), lr=0.01)

for epoch in range(150):
    opt_s2.zero_grad()
    recon = model_s2(X_noisy_train)
    loss = nn.functional.mse_loss(recon, X_clean_train)
    loss.backward()
    opt_s2.step()

```

10.2.3 Stage 3 — Joint Fine-Tuning (Optional)

After stage 2, unfreeze both encoder and decoder for a short joint fine-tuning pass:

```

for p in decoder_layer.parameters():
    p.requires_grad = True

opt_s3 = torch.optim.Adam(model_s2.parameters(), lr=0.001)
for epoch in range(50):
    opt_s3.zero_grad()
    recon = model_s2(X_noisy_train)
    loss = nn.functional.mse_loss(recon, X_clean_train)
    loss.backward()
    opt_s3.step()

```

10.3 Why Phase Information Matters

The SWAP test measures $|\langle\psi|\phi\rangle|^2$, which is *phase-insensitive*: two states that differ only by a global phase appear identical. Direct qubit measurements are also phase-insensitive.

However, the *relative* phase between amplitudes affects interference patterns in multi-qubit circuits. If we use `qml.state()` measurements or amplitude-based loss functions, phase information is preserved. The Mirror QAE, by using the adjoint decoder, maintains phase coherence between encoder and decoder by construction.

10.4 Comparison: Pure vs Hybrid vs Mirror QAE

Criterion	Pure PennyLane	Hybrid PyTorch	Mirror QAE
Classical layers	None	Pre + post	None (structured training)
Training protocol	Joint	Joint	3-stage
Phase preservation	Partial	No	Yes
Implementation complexity	Low	Medium	High
Convergence stability	Low	Medium	High

10.5 Exercises

1. Implement the full three-stage Mirror QAE. Compare its test MSE to the naive joint-training QAE. Does staged training produce a lower final MSE?
2. In the Hybrid QAE, vary the `n_latent` compression ratio. Plot reconstruction MSE vs compression ratio for both clean and noisy inputs.
3. Add learning rate decay to Stage 2 training: `scheduler = torch.optim.lr_scheduler.StepLR(opt_s2, step_size=50, gamma=0.5)`. Does it improve convergence?
4. Add noise at different levels (`noise_level` `{0.01, 0.05, 0.1, 0.2}`) and compare how pure QAE and Hybrid QAE handle increasing noise.
5. Replace `StronglyEntanglingLayers` with `BasicEntanglerLayers` in the Mirror QAE. Does the 3-stage protocol still produce good reconstructions?
6. The adjoint decoder assumes the encoder is unitary. Classical layers break this assumption. Discuss: what does “latent space” mean in the Hybrid QAE, and how does it differ from the pure quantum case?

Part IV

Appendices

Chapter 11

PennyLane API Reference

Quick reference for the PennyLane APIs used in this book. See the official documentation at <https://docs.pennylane.ai> for full details.

11.1 Devices

```
qml.device("default.qubit", wires=n) # Pure Python simulator
qml.device("lightning.qubit", wires=n) # C++ simulator (fast)
qml.device("lightning.gpu", wires=n) # GPU simulator (requires CUDA)
```

11.2 QNode Decorator

```
@qml.qnode(dev, diff_method='adjoint') # fastest for most circuits
@qml.qnode(dev, diff_method='parameter-shift') # required for reuploading
@qml.qnode(dev, interface='torch') # enables PyTorch autograd
```

diff_method	Speed	Supports reuploading
'adjoint'	Fast	No
'parameter-shift'	Slow (2 evals/param)	Yes
'backprop'	Fast (requires default.qubit)	Yes

11.3 Single-Qubit Gates

Gate	PennyLane	Matrix
Pauli-X (NOT)	<code>qml.PauliX(wire)</code>	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$
Pauli-Y	<code>qml.PauliY(wire)</code>	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$
Pauli-Z	<code>qml.PauliZ(wire)</code>	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$

Gate	PennyLane	Matrix
Hadamard	<code>qml.Hadamard(wire)</code>	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$
$R_X(\theta)$	<code>qml.RX(theta, wire)</code>	$e^{-i\theta X/2}$
$R_Y(\theta)$	<code>qml.RY(theta, wire)</code>	$e^{-i\theta Y/2}$
$R_Z(\theta)$	<code>qml.RZ(theta, wire)</code>	$e^{-i\theta Z/2}$

11.4 Two-Qubit Gates

```
qml.CNOT(wires=[control, target])
qml.CZ(wires=[control, target])
qml.SWAP(wires=[wire0, wire1])
qml.CRY(theta, wires=[control, target])
qml.CSWAP(wires=[control, wire0, wire1]) # Fredkin gate
```

11.5 Data Encoding

```
qml.AngleEmbedding(features, wires, rotation='Y')
# features: (n_qubits,) - one angle per qubit

qml.AmplitudeEmbedding(features, wires, normalize=True)
# features: (2^n_qubits,) - full amplitude vector

qml.BasisEmbedding(features, wires)
# features: binary list/array of length n_qubits
```

11.6 Ansatz Templates

```
qml.BasicEntanglerLayers(weights, wires, rotation=qml.RY)
# weights shape: (n_layers, n_wires)

qml.StronglyEntanglingLayers(weights, wires)
# weights shape: (n_layers, n_wires, 3)
```

11.7 Measurements

```
qml.probs(wires) # probability vector, shape (2^n,)
qml.expval(observable) # scalar [-1, +1]
qml.state() # full statevector, shape (2^n,)
qml.sample(observable) # raw samples
```

Common observables:

```
qml.PauliZ(wire)
qml.PauliX(wire)
qml.Hermitian(matrix, wires)
```

11.8 Visualisation

```
print(qml.draw(circuit)(args))          # ASCII diagram
qml.draw_mpl(circuit)(args)             # matplotlib figure
qml.draw_mpl(circuit, decimals=2)(args) # rounded parameter values
```

11.9 Differentiation

```
# Gradient of scalar-output circuit
grad_fn = qml.grad(circuit)
grads    = grad_fn(params, x)

# Jacobian for vector-output circuit
jac = qml.jacobian(circuit)(params)

# Adjoint of a unitary template
qml.adjoint(qml.StronglyEntanglingLayers)(weights, wires=wires)
```

11.10 PyTorch Integration

```
# 1. QNode with torch interface
@qml.qnode(dev, interface="torch", diff_method="adjoint")
def circuit(inputs, weights):
    ...

# 2. Wrap as nn.Module layer
weight_shapes = {"weights": (n_layers, n_qubits, 3)}
layer = qml.qnn.TorchLayer(circuit, weight_shapes)

# 3. Use in nn.Module
class MyModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.q = layer
        self.out = nn.Linear(1, 1)

    def forward(self, x):
        return self.out(self.q(x).float().unsqueeze(-1))
```

11.11 Native Optimisers (PennyLane only)

```
opt = qml.AdamOptimizer(stepsize=0.05)
opt = qml.GradientDescentOptimizer(stepsize=0.01)
opt = qml.RMSPropOptimizer(stepsize=0.01)
opt = qml.NesterovMomentumOptimizer(stepsize=0.01, momentum=0.9)

# One step
params = opt.step(cost_fn, params)

# One step + cost value
params, cost_val = opt.step_and_cost(cost_fn, params)
```

Chapter 12

Mathematical Background

This appendix collects the mathematical tools used in the book. It is not a substitute for a quantum mechanics course; rather, it provides just enough formalism to read the code and proofs.

12.1 Linear Algebra Essentials

12.1.1 Vectors and Inner Products

A quantum state of n qubits is a unit vector in $\mathbb{C}^{2^n}$:

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle, \quad \sum_i |\alpha_i|^2 = 1$$

The *inner product* of two states $|\psi\rangle$ and $|\phi\rangle$:

$$\langle\psi|\phi\rangle = \sum_i \alpha_i^* \beta_i \in \mathbb{C}$$

The *fidelity* (overlap squared):

$$F(\psi, \phi) = |\langle\psi|\phi\rangle|^2 \in [0, 1]$$

12.1.2 Tensor Products

For two-qubit systems, the basis states are tensor products:

$$|00\rangle = |0\rangle \otimes |0\rangle, \quad |01\rangle = |0\rangle \otimes |1\rangle, \quad \dots$$

The state of a product (non-entangled) system:

$$|\psi_A\rangle \otimes |\psi_B\rangle$$

12.1.3 Density Matrices

A *pure state* $|\psi\rangle$ can equivalently be described by its density matrix:

$$\rho = |\psi\rangle\langle\psi|$$

For a system with subsystems A (latent) and B (trash), the *partial trace* gives the reduced state of A :

$$\rho_A = \text{Tr}_B[\rho]$$

12.2 Quantum Gates as Matrices

Single-qubit gates act on $\mathbb{C}^2$; they are 2×2 unitary matrices.

12.2.1 Rotation Gates

$$R_X(\theta) = e^{-i\theta X/2} = \begin{pmatrix} \cos(\theta/2) & -i \sin(\theta/2) \\ -i \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$$

$$R_Y(\theta) = e^{-i\theta Y/2} = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$$

$$R_Z(\theta) = e^{-i\theta Z/2} = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix}$$

Applied to $|0\rangle$:

$$\begin{aligned} R_Y(\theta)|0\rangle &= \cos\frac{\theta}{2}|0\rangle + \sin\frac{\theta}{2}|1\rangle \\ \Rightarrow P(|0\rangle) &= \cos^2(\theta/2), \quad P(|1\rangle) = \sin^2(\theta/2) \end{aligned}$$

12.3 Measurements and Observables

12.3.1 Expectation Value

For observable $\hat{O}$ and state $|\psi\rangle$:

$$\langle\hat{O}\rangle = \langle\psi|\hat{O}|\psi\rangle$$

For PauliZ on qubit 0:

$$\langle Z_0 \rangle = P(|0\rangle) - P(|1\rangle) \in [-1, +1]$$

12.3.2 Pauli Operators

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

12.4 Variational Principle

A *variational circuit* parameterises a unitary:

$$U(\theta) = \prod_l U_l(\theta_l)$$

where each $U_l(\theta_l) = e^{-i\theta_l H_l/2}$ for some Hermitian generator H_l .

The model output is:

$$f(\mathbf{x}; \theta) = \langle 0 | U^\dagger(\theta) \phi^\dagger(\mathbf{x}) \hat{O} \phi(\mathbf{x}) U(\theta) | 0 \rangle$$

12.5 Parameter-Shift Rule

For generators with two eigenvalues $\pm r$:

$$\frac{\partial f}{\partial \theta_l} = r \left[f\left(\theta_l + \frac{\pi}{4r}\right) - f\left(\theta_l - \frac{\pi}{4r}\right) \right]$$

For standard Pauli rotations ($r = 1/2$, shift = $\pi/2$):

$$\frac{\partial f}{\partial \theta_l} = \frac{f(\theta_l + \pi/2) - f(\theta_l - \pi/2)}{2}$$

This is exact (not an approximation) and requires exactly 2 circuit evaluations per parameter.

12.6 Loss Functions

12.6.1 Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Used for regression (Chapters 4–5).

12.6.2 R^2 (Coefficient of Determination)

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

$R^2 \in (-\infty, 1]$; $R^2 = 1$ is perfect fit.

12.6.3 Binary Cross-Entropy (BCE)

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

Used for classification (Chapters 6–8). Requires $\hat{p}_i \in (0, 1)$.

12.6.4 Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$$

Maps logits (real values) to probabilities.

12.7 SWAP Test

The SWAP test circuit estimates fidelity without direct state access:

$$P(\text{ancilla} = |0\rangle) = \frac{1 + |\langle\psi|\phi\rangle|^2}{2}$$

$$\Rightarrow F(\psi, \phi) = |\langle\psi|\phi\rangle|^2 = 2P(|0\rangle_{\text{anc}}) - 1$$

12.8 Mackey-Glass Equation

$$\frac{dx(t)}{dt} = \frac{\beta x(t - \tau)}{1 + x(t - \tau)^n} - \gamma x(t)$$

Parameter	Typical value	Role
β	0.2	Positive feedback strength
γ	0.1	Decay rate
τ	17	Delay (controls chaos)
n	10	Nonlinearity exponent

For $\tau > 16.8$, the system exhibits chaotic dynamics. For $\tau \leq 16.8$, it converges to a limit cycle.

Chapter 13

Datasets

13.1 UCI Automobile Dataset

Used in: Chapters 6 and 7 (classification).

Source: UCI Machine Learning Repository — <https://archive.ics.uci.edu/dataset/10/automobile>

Description: Technical specifications and insurance risk ratings for 205 automobile models from the 1985 Ward's Automotive Yearbook.

Target variable: `symboling` — an integer in $[-3, 3]$ representing the degree to which the automobile is more risky than its price indicates. We convert to a binary label:

Symboling	Meaning	Binary label
≤ 0	Safe (or safe for price)	0
> 0	Riskier than price suggests	1

Key numeric features used:

Feature	Description
<code>wheel-base</code>	Distance between axles (inches)
<code>length, width, height</code>	Body dimensions
<code>curb-weight</code>	Weight of car without passengers
<code>engine-size</code>	Engine displacement (cubic inches)
<code>horsepower</code>	Engine power
<code>city-mpg, highway-mpg</code>	Fuel efficiency
<code>price</code>	Price in USD

Loading:

```
import pandas as pd
import numpy as np

df = pd.read_csv('../data/Automobile.csv')
```

```
# Keep numeric columns, drop missing values
df_num = df.select_dtypes(include=[np.number]).dropna()

y = (df_num['symboling'] > 0).astype(float).values
X = df_num.drop(columns=['symboling']).values

print(f"Samples: {len(X)}, Features: {X.shape[1]}, Positive class: {y.mean():.1%}")
```

Typical output: ~160 samples, 13 features, ~60% positive class.

13.2 Mackey-Glass Time Series

Used in: Chapters 9 and 10 (quantum autoencoder).

Description: A synthetic time series generated by the Mackey-Glass delay-differential equation (see Chapter 12). It serves as a standard benchmark for time-series prediction and denoising.

Generation:

```
def mackey_glass(n_samples, tau=17, beta=0.2, gamma=0.1,
                 n=10, dt=1.0, x0=0.9):
    """
    Generate a Mackey-Glass time series using Euler integration.

    Parameters
    -----
    n_samples : int
        Number of time steps to generate.
    tau : int
        Delay parameter. Values > 16.8 produce chaotic dynamics.
    beta, gamma : float
        Feedback strength and decay rate.
    n : int
        Nonlinearity exponent.
    dt : float
        Integration step size.
    x0 : float
        Initial condition.

    Returns
    -----
    np.ndarray of shape (n_samples,)
    """
    history = np.full(tau + 1, x0)
    x = list(history)
    for _ in range(n_samples):
        x_delayed = x[-tau]
```

```

    dx = beta * x_delayed / (1 + x_delayed**n) - gamma * x[-1]
    x.append(x[-1] + dt * dx)
    return np.array(x[tau:])

```

Noise injection:

```

def add_noise(ts, noise_level=0.05, distribution='normal', seed=None):
    """
    Add noise to a time series.

    Parameters
    -----
    noise_level : float
        Noise standard deviation as a fraction of ts.std().
    distribution : {'normal', 'uniform'}
    """
    rng = np.random.default_rng(seed)
    sigma = noise_level * ts.std()
    if distribution == 'normal':
        noise = rng.normal(0, sigma, size=ts.shape)
    else:
        noise = rng.uniform(-sigma, sigma, size=ts.shape)
    return ts + noise

```

Typical usage:

```

ts_clean = mackey_glass(n_samples=1000, tau=17)
ts_noisy = add_noise(ts_clean, noise_level=0.10)

# Train/test split (temporal - no shuffling)
split = int(0.8 * len(ts_clean))
ts_train_clean = ts_clean[:split]
ts_test_clean = ts_clean[split:]
ts_train_noisy = ts_noisy[:split]
ts_test_noisy = ts_noisy[split:]

```

Warning

Do not shuffle time series data before splitting. Shuffling breaks temporal structure and leads to data leakage (future time steps in the training set).

13.3 Scikit-learn Synthetic Datasets

Used in: Chapter 5 (quantum estimator).

13.3.1 make_regression

```
from sklearn.datasets import make_regression

X, y = make_regression(
    n_samples=100,
    n_features=1,      # number of informative features
    noise=10.0,        # Gaussian noise on y
    random_state=42
)
```

13.3.2 make_swiss_roll

```
from sklearn.datasets import make_swiss_roll

X_3d, t = make_swiss_roll(n_samples=200, noise=0.1, random_state=42)
X = X_3d[:, [0, 2]]  # 2D projection
y = t                # colour coordinate (continuous target)
```

The Swiss Roll is a 2D manifold embedded in 3D. Its nonlinear structure tests whether the quantum estimator can learn beyond linear relationships.

References